

# **Computer Vision Lab Manual**

Complete Sequential Compilation with Associated Source File Names

## Practical 1: Basic Geometric Drawing Operations on Images

**Objective:** Learn how to read an image and perform basic geometric drawing operations (drawing lines, rectangles, circles, and putting text) using OpenCV.

 File: pr1.py

```
import cv2

# Read the input image
img = cv2.imread('image1.png')

# Line
cv2.line(img, (0, 0), (200, 200), (255, 0, 0), 3)

# Rectangle
cv2.rectangle(img, (50, 50), (150, 150), (0, 255, 0), 2)

# Circle
cv2.circle(img, (100, 100), 50, (0, 0, 255), -1)

# Text
cv2.putText(img, 'OpenCV', (10, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255),
2)
```

## Practical 2: Image Color Space Transformations & Contour Tracking

**Objective:** Perform color space shifts, standard static image thresholding, find contours of elements, and visualize structural outlines.

 File: pr1(2).py

```
import cv2

image =cv2.imread("C:/Users/Admin/Desktop/CV PR/flower.png")
cv2.imshow("original image",image)
cv2.waitKey(0)

image_grey=cv2.cvtColor(image,cv2.COLOR_BGR2GRAY)
cv2.imshow("gray image",image_grey)
cv2.waitKey(0)

ret, thresh=cv2.threshold(image_grey, 150, 255,cv2.THRESH_BINARY)
cv2.imshow("Binary Image",thresh)
cv2.waitKey(0)

contours, heirarchy=cv2.findContours(thresh,cv2.RETR_TREE, cv2.CHAIN_APPROX_NONE)
image_copy=image.copy()

cv2.drawContours(image_copy, contours, -1, (0,0,255),2, cv2.LINE_AA)
cv2.imshow("contour Image",image_copy)
cv2.waitKey(0)
```

## Practical 3: Pixel Intensity Distribution & Histogram Analysis

**Objective:** Flatten an image grid to extract its quantitative pixel intensity histogram and plot the Cumulative Distribution Function (CDF).

 **File:** pr4.py

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Load sample image in grayscale configuration
img = cv2.imread("flower_bw.png")

# Calculate the distribution frequency of intensities across 256 structural bins
hist, bins = np.histogram(img.flatten(), 256, [0, 256])

# Compute Cumulative Distribution Function (CDF)
cdf = hist.cumsum()
cdf_normalized = cdf * hist.max() / cdf.max()

# Plot analytics layout using matplotlib components
plt.plot(cdf_normalized, color='r')
plt.hist(img.flatten(), 256, [0, 256], color='g')
plt.xlim([0, 256])
plt.ylabel('Pixel Count')
plt.legend(('cdf', 'histogram'), loc='upper left')
plt.show()
```

## Practical 4: Pyramidal Mean Shift Image Segmentation (OpenCV Native)

**Objective:** Smooth out fine details and segment local color regions using OpenCV's native Pyramidal Mean Shift filtering function.

 **File:** pr3.py

```
# Mean shift segmentation in Python
import cv2
import numpy as np

# Loading the image
image = cv2.imread("image4.jpg")

# Converting image from BGR to RGB
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Applying pyramidal mean shift filtering
# sp: spatial window radius, sr: color/range window radius
seg_image = cv2.pyrMeanShiftFiltering(image, sp=20, sr=30)

# Convert back to BGR for display
seg_image_bgr = cv2.cvtColor(seg_image, cv2.COLOR_RGB2BGR)

# Showing segmented and original image
cv2.imshow("Original Image", cv2.cvtColor(image, cv2.COLOR_RGB2BGR))
cv2.imshow("Mean shift segmentation using OpenCV only", seg_image_bgr)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

## Practical 5: Clustered Mean Shift Image Segmentation (Scikit-Learn)

**Objective:** Flatten image vectors to estimate automated color distribution bandwidth fields and perform non-parametric Mean Shift clustering.

 **File: PR3(2).py**

```
import cv2
import numpy as np
from sklearn.cluster import MeanShift, estimate_bandwidth

# Loading the image
image = cv2.imread("image4.jpg")
# Converting BGR to RGB
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Converting image into a flat 2D array (pixels x color channels)
pixels = image.reshape(-1, 3)

# Estimating automatic bandwidth kernel distribution parameters
bandwidth = estimate_bandwidth(pixels, quantile=0.1, n_samples=1000)

# Execute MeanShift clustering from Scikit-Learn
ms = MeanShift(bandwidth=bandwidth, bin_seeding=True)
ms.fit(pixels)

# Get labels assigned to pixels and cluster centers
labels = ms.labels_
cluster_centers = ms.cluster_centers_

# Replacing each pixel value with its corresponding cluster center color
segmented_pixels = cluster_centers[labels]

# Reshape pixel array back into original dimensional layout matrix
segmented_image = segmented_pixels.reshape(image.shape).astype(np.uint8)

# Convert back to native BGR format for display visualization
seg_image_bgr = cv2.cvtColor(segmented_image, cv2.COLOR_RGB2BGR)

cv2.imshow("Original Image", cv2.cvtColor(image, cv2.COLOR_RGB2BGR))
cv2.imshow("Segmented Image using Scikit-learn", seg_image_bgr)
```

```
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

## Practical 6: Bounding-Box Graph Cut Foreground Image Segmentation

**Objective:** Isolate a focal foreground subject from background components within defined bounding box coordinates using the GrabCut algorithm.

 **File:** pr2.py

```
import cv2
import numpy as np
from matplotlib import pyplot as plt
def apply_cut(image_path,react):
    img=cv2.imread(image_path)
    img_rgb=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
    mask = np.zeros(img.shape[:2], np.uint8)

    bgd_model=np.zeros((1,65),np.float64)
    fgd_model=np.zeros((1,65),np.float64)

    cv2.grabCut(img_rgb,mask,react,bgd_model,fgd_model,5,cv2.GC_INIT_WITH_RECT)
    mask2=np.where((mask==2)|(mask==0),0,1).astype('uint8')
    img_segmented=img_rgb*mask2[:, :, np.newaxis]
    return img_rgb,img_segmented

rectangle=(50,50,400,300)
original_image,segmented_image=apply_cut('C:/Users/Admin/Desktop/CV PR/
flower.png',rectangle)

plt.figure(figsize=(10,8))
plt.subplot(121),plt.imshow(original_image)
plt.title("original image") ,plt.xticks([]),plt.yticks([])
plt.subplot(122),plt.imshow(segmented_image)
plt.title("segmented image graph cut"),plt.xticks([]),plt.yticks([])
plt.show()
```

## Practical 7: Convolutional Spatial Filters & Gradient Edge Detection

**Objective:** Implement spatial gradient directional operators (Sobel X, Sobel Y, Sobel XY) to extract variations across region transitions.

 **File: PR5.py**

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("RUL-lung-nodule_Pina.png")
# cv2.imshow("Original Image", image)

image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
image_blur = cv2.GaussianBlur(image_gray, (5,5), 0)

# sobel Edge Detection
sobelx = cv2.Sobel(src=image_blur, ddepth=cv2.CV_64F, dx=1, dy=0, ksize=3) # sobel
edge detection on x axis
sobely = cv2.Sobel(src=image_blur, ddepth=cv2.CV_64F, dx=0, dy=1, ksize=3) # sobel
edge detection on y axis
sobelxy = cv2.Sobel(src=image_blur, ddepth=cv2.CV_64F, dx=1, dy=1, ksize=5) # on
both the axis

# canny edge detection
# edge = cv2.Canny(image_gray, 50, 150)

# display sobel edge detection
image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
sobelx = cv2.convertScaleAbs(sobelx)
sobely = cv2.convertScaleAbs(sobely)
sobelxy = cv2.convertScaleAbs(sobelxy)

sobelx_rgb = cv2.cvtColor(sobelx, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(15, 8))
plt.subplot(1,4,1)
plt.imshow(image_gray)
plt.title('Original')
plt.axis('off')

plt.subplot(1,4,2)
plt.imshow(sobelx, cmap='gray')
```

```
plt.title('SOBEL X')
plt.axis('off')

plt.subplot(1,4,3)
plt.imshow(sobely, cmap='gray')
plt.title('SOBEL Y')
plt.axis('off')

plt.subplot(1,4,4)
plt.imshow(sobelxy, cmap='gray')
plt.title('SOBEL X and Y')
plt.axis('off')
plt.show()
```

## Practical 8: Chromatic Threshold Tracking (HSV Color Masking)

**Objective:** Target specifically localized features matching explicit hue boundaries (e.g., Yellow features) by migrating into the HSV spectrum.

 **File: PR5(2).py**

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def detect_yellow(image_path):
    # load the image
    img = cv2.imread(image_path)

    # convert to HSV
    hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

    yellow_lower = np.array([20, 100, 100])
    yellow_upper = np.array([30, 255, 255])

    # create a mask to isolate yellow color
    mask = cv2.inRange(hsv_img, yellow_lower, yellow_upper)
    # mask = cv2.inRange(hsv_img, green_lower, green_upper)

    # find contours of yellow color
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)

    output_img = img.copy()

    # loop through each contour and draw bounding box
    for contour in contours:
        x,y,w,h = cv2.boundingRect(contour)
        if cv2.contourArea(contour) > 500:
            cv2.rectangle(output_img, (x,y), (x+w, y+h), (0,0,255), 3)

    # convert image to RGB
    output_img_rgb = cv2.cvtColor(output_img, cv2.COLOR_BGR2RGB)
    #display
    plt.figure(figsize=(12,6))

    # original image
```

```
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis('off')
# processed image
plt.subplot(1,2,2)
plt.imshow(output_img_rgb)
plt.title("Yellow color detection on Image")
plt.axis('off')
plt.show()

return output_img

image_path = 'fruits.png'
output_image = detect_yellow(image_path)
```

## Practical 9: Sequential Particle Filter Target Tracking Optimization

**Objective:** Track user-selected regions across sequential video frames using particle distributions and mean distance estimations.

 **File: PR6.py**

```
import cv2, numpy as np

cap = cv2.VideoCapture(0)
N = 800

ret, frame = cap.read()
x, y, w, h = cv2.selectROI(frame, False)

roi = frame[y:y+h, x:x+w]
ref_color = roi.mean(axis=(0,1))

particles = np.tile([x+w/2, y+h/2], (N,1))
weights = np.ones(N) / N

while True:
    ret, frame = cap.read()
    if not ret: break

    h_img, w_img = frame.shape[:2]

    # Predict
    particles += np.random.randn(N,2) * 15

    # Keep particles inside frame
    particles[:,0] = np.clip(particles[:,0], 0, w_img-1)
    particles[:,1] = np.clip(particles[:,1], 0, h_img-1)

    # Measurement (vectorized)
    coords = particles.astype(int)
    colors = frame[coords[:,1], coords[:,0]]
    dist = np.linalg.norm(colors - ref_color, axis=1)
    weights = 1 / (dist + 1)

    weights /= weights.sum()
```

```

# Estimate
pos = np.average(particles, weights=weights, axis=0).astype(int)

# Resample
idx = np.random.choice(N, N, p=weights)
particles = particles[idx]
weights.fill(1.0 / N)

# Draw
for p in particles:
    cv2.circle(frame, tuple(p.astype(int)), 1, (0,255,0), -1)
cv2.circle(frame, tuple(pos), 8, (0,0,255), -1)

cv2.imshow("Particle Filter", frame)
if cv2.waitKey(30) & 0xFF == 27:
    break

cap.release()
cv2.destroyAllWindows()

```

## Practical 10: Real-Time Hand Gesture & Finger Tracking Recognition

**Objective:** Leverage structural node graph evaluation systems to dynamically translate human fingers and landmarks into commands.

 **File: PR7.py**

```
import cv2
import mediapipe as mp
from mediapipe.tasks.python import vision
from mediapipe.tasks.python.core.base_options import BaseOptions

# Load the hand landmark model (download required)
model_path = "hand_landmarker.task"

# Initialize HandLandmarker
options = vision.HandLandmarkerOptions(
    base_options=BaseOptions(model_asset_path=model_path),
    num_hands=1
)
hand_landmarker = vision.HandLandmarker.create_from_options(options)

# Gesture recognition function
def recognize_number(hand_landmarks):
    fingers = []
    # Landmark shortcuts
    lm = hand_landmarks
    # ---- Thumb (special case: compare x instead of y) ----
    if lm[4].x < lm[3].x:
        fingers.append(1)
    else:
        fingers.append(0)
    # ---- Other 4 fingers ----
    tips = [8, 12, 16, 20]
    for tip in tips:
        if lm[tip].y < lm[tip - 2].y:
            fingers.append(1)
        else:
            fingers.append(0)

    total_fingers = sum(fingers)
```

```

# ---- Map to numbers ----
if fingers == [0,0,0,0,0]:
    return "0"
elif fingers == [0,1,0,0,0]:
    return "1"
elif fingers == [0,1,1,0,0]:
    return "2"
elif fingers == [0,1,1,1,0]:
    return "3"
elif fingers == [0,1,1,1,1]:
    return "4"
elif fingers == [1,1,1,1,1]:
    return "5"
else:
    return str(total_fingers)

# Start webcam
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    frame = cv2.flip(frame, 1) # flipping horizontaly
    if not ret:
        break

    # Convert frame to RGB
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    # Convert to MediaPipe Image
    mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb_frame)

    # Detect hands
    result = hand_landmarker.detect(mp_image)

    if result.hand_landmarks:
        for hand_landmarks in result.hand_landmarks:
            gesture = recognize_number(hand_landmarks)

            # Draw landmarks manually
            for lm in hand_landmarks:
                h, w, _ = frame.shape
                cx, cy = int(lm.x * w), int(lm.y * h)
                cv2.circle(frame, (cx, cy), 5, (0, 255, 0), -1)

```

```

        # Get position of wrist (landmark 0)
        h, w, _ = frame.shape
        x = int(hand_landmarks[0].x * w)
        y = int(hand_landmarks[0].y * h)

        cv2.putText(frame, gesture, (x, y - 20),
                    cv2.FONT_HERSHEY_SIMPLEX, 1,
                    (0, 0, 0), 2)

    cv2.imshow("Hand Gesture Recognition", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

```

## Practical 11: 3D Scatter Graph Projection Plots

**Objective:** Formulate multidimensional mathematical data graphs inside a structured 3D projection plane using numpy arrays.

 **File: PR8.py**

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

# creating figuresize
fig = plt.figure(figsize=(10,8))

# generating a 3d sine wave
ax = plt.axes(projection='3d')

# creating axes
axes = [5,5,5]

# create data
data = np.ones(axes)

# control transparency
alpha = 0.9

# Control colour
colors = np.empty(axes + [4])

# Creating array points using numpy
x = np.arange(0, 20, 0.1)
y = np.cos(x)
z = y*np.sin(x)
c = x + y

# To create a scatter graph
ax.scatter(x, y, z, c=c)

# turn off/on axis
plt.axis('off')
plt.show()
```

## Practical 12: Mathematical Frequency Filtering & Image Noise Suppression

**Objective:** Compare performance attributes of Mean, Gaussian, and Median filters on digital images corrupted with noise patterns.

 **File: PR9.py**

```
import cv2
from matplotlib import pyplot as plt

image = cv2.imread('noisysalterpepper.png')

# Apply mean filter (kernel size 3x3)
mean_blur = cv2.blur(image, (3,3)) # it is suitable for uniform noise

# Apply Gaussian smoothing
gaussian_blur = cv2.GaussianBlur(image, (3,3), 0) # it is suitable for gaussian noise

# Apply median filter
median_blur = cv2.medianBlur(image, 5) # it is suitable for salt and pepper noise (black and white dots)

# Displaying results
plt.figure(figsize=(10, 8))
plt.subplot(2,2,1)
plt.imshow(image)
plt.title("Original")
plt.axis('off')

plt.subplot(2,2,2)
plt.imshow(mean_blur)
plt.title("Mean Filter")
plt.axis('off')

plt.subplot(2,2,3)
plt.imshow(gaussian_blur)
plt.title("Gaussian Filter")
plt.axis('off')

plt.subplot(2,2,4)
```

```
plt.imshow(median_blur)
plt.title("Median Filter")
plt.axis('off')
plt.tight_layout()
plt.show()
```

## Practical 13: Adaptive Threshold Segmentation & Object Count Analytics

**Objective:** Use adaptive thresholding and morphological filters to count distinct isolated object targets under non-uniform illumination.

 **File:** PR10.py

```
import cv2
import numpy as np

image = cv2.imread('objects2.png')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Blur (important)
blur = cv2.GaussianBlur(gray, (11,11), 0)

# Adaptive threshold (handles lighting better)
thresh = cv2.adaptiveThreshold(
    blur, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY_INV,
    11, 2
)

# Morphological closing (fills gaps inside coins)
kernel = np.ones((5,5), np.uint8)
closing = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel, iterations=3)

# Remove small noise
opening = cv2.morphologyEx(closing, cv2.MORPH_OPEN, kernel, iterations=2)

# Find contours
contours, _ = cv2.findContours(opening, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

count = 0

for cnt in contours:
    area = cv2.contourArea(cnt)

    # Tune this threshold based on your image
    if area > 1000:
```

```
        count += 1
        cv2.drawContours(image, [cnt], -1, (0,255,0), 3)

print("Number of Objects:", count)

cv2.imshow("Threshold", opening)
cv2.imshow("Result", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```